

開發 Spring boot 應用程式-後端

讓我們首先建一個 Spring boot 應用程式並構建一個簡單的 REST API。

1. 建 Spring Boot 應用程式

建 Spring Boot 應用程式的方法有很多種。您可以參考以下文章來建 Spring Boot 應用程式。

2. 添加 maven 依賴

這是一個完整的 *pom.xml* 文件供您參考：

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
https://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>
  <parent>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-parent</artifactId>
    <version>3.0.4</version>
    <relativePath/>
  </parent>
  <groupId>net.javaguides</groupId>
  <artifactId>springboot-backend</artifactId>
  <version>0.0.1-SNAPSHOT</version>
  <name>springboot-backend</name>
  <description>Spring Boot backend application</description>
  <properties>
    <java.version>17</java.version>
  </properties>
  <dependencies>
    <dependency>
      <groupId>org.springframework.boot</groupId>
```

```
<artifactId>spring-boot-starter-data-jpa</artifactId>
  </dependency>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>
  </dependency>

  <dependency>
    <groupId>com.h2database</groupId>
    <artifactId>h2</artifactId>
    <scope>runtime</scope>
  </dependency>
  <dependency>
    <groupId>org.projectlombok</groupId>
    <artifactId>lombok</artifactId>
    <optional>true</optional>
  </dependency>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-test</artifactId>
    <scope>test</scope>
  </dependency>
</dependencies>

<build>
  <plugins>
    <plugin>

<groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-maven-plugin</artifactId>
    <configuration>
      <excludes>
        <exclude>

<groupId>org.projectlombok</groupId>

<artifactId>lombok</artifactId>
```

```
        </exclude>
    </excludes>
</configuration>
</plugin>
</plugins>
</build>

</project>
```

3.建 JPA 實體-Employee.java

建具有以下內容的 *Employee* 類別 -

```
package model;

import lombok.*;

import jakarta.persistence.*;

@Getter
@Setter
@NoArgsConstructor
@AllArgsConstructor
@Builder
@Entity
@Table(name = "employees")
public class Employee {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private long id;

    @Column(name = "first_name", nullable = false)
    private String firstName;

    @Column(name = "last_name")
    private String lastName;
```

```
        private String email;

    }
}
```

4. 建 Spring Data JPA 存儲庫 - EmployeeRepository.java

我們建一個 Spring Data JPA 存儲庫來與 H2 程式庫通信。
讓我們建一個名為 *repository* 的套件，然後在 *repository* 中建以下 *EmployeeRepository* 介面-

```
import org.springframework.data.jpa.repository.JpaRepository;

public interface EmployeeRepository extends JpaRepository<Employee, Long> {

}
```

5.帶有 REST API 的 Spring 控制器 - /api/employees

讓我們 建一個套件，然後在 控制器中創建以下 *EmployeeController* 類別，內容如下 -

```
package net.javaguides.springboot.controller;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.*;

import java.util.List;

@RestController
@RequestMapping("/api")
@CrossOrigin("http://localhost:3000/")
public class EmployeeController {

    @Autowired
```

```
private EmployeeRepository employeeRepository;

@GetMapping("/employees")
public List<Employee> fetchEmployees(){
    return employeeRepository.findAll();
}
}
```

請注意，我們添加了以下代碼行以避免 CORS 問題：

```
@CrossOrigin(origins = "http://localhost:3000")
```

6. 運行 Spring Boot 應用程序並測試 Rest API

讓我們在應用程序啟動時在員工表中插入一些記錄。

```
package net.javaguides.springboot;

import net.javaguides.springboot.entity.Employee;
import net.javaguides.springboot.repository.EmployeeRepository;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.CommandLineRunner;
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class SpringbootBackendApplication implements CommandLineRunner {

    public static void main(String[] args) {
        SpringApplication.run(SpringbootBackendApplication.class, args);
    }

    @Autowired
    private EmployeeRepository employeeRepository;

    @Override
    public void run(String... args) throws Exception {
```

```
Employee employee1 = Employee.builder()
    .firstName("Ramesh")
    .lastName("Fadatare")
    .email("ramesh@gmail.com")
    .build();

Employee employee2 = Employee.builder()
    .firstName("Tony")
    .lastName("Stark")
    .email("tony@gmail.com")
    .build();

Employee employee3 = Employee.builder()
    .firstName("John")
    .lastName("Cena")
    .email("cena@gmail.com")
    .build();

employeeRepository.save(employee1);
employeeRepository.save(employee2);
employeeRepository.save(employee3);
    }
}
```

讓我們從 IDE -> 右鍵單擊 -> Run As -> Java Application 運行這個 Spring Boot 應用程式：

在瀏覽器中點擊 <http://localhost:8080/api/employees>

```
[
  {
    "id":1,
    "firstName":"Ramesh",
    "lastName":"Fadatare",
    "email":"ramesh@gmail.com"
  },
  {
```

```
    "id":2,
    "firstName":"Tony",
    "lastName":"Stark",
    "email":"tony@gmail.com"
  },
  {
    "id":3,
    "firstName":"John",
    "lastName":"Cena",
    "email":"cena@gmail.com"
  }
]
```

2. 建 React JS 前端應用程序

讓我們繼續建一個 React 應用程序來使用 `/api/employees` REST API。

我們將使用 VS Code IDE 來構建 React 應用程序。

我們將使用 React Hook (useState) 在功能組件中使用狀態。

1 - 使用 Create React App 建 React UI

Create **React App** CLI 工具是官方支持的創建單頁 React 應用程序的方法。它提供了無需配置的現代構建設置。

要建新的 React 應用程序，請在 VS Code 中打開終端並鍵入以下命令：

```
npx create-react-app react-frontend
```

運行這些命令中的任何一個都會在當前文件夾中建一個名為 `react-frontend` 的目錄。在該目錄中，它將生成初始項目結構並安裝傳遞依賴項：

```
react-frontend
├─ README.md
├─ node_modules
├─ package.json
├─ .gitignore
├─ public
│  └─ favicon.ico
│  └─ index.html
```

```
|   ├── logo192.png
|   ├── logo512.png
|   ├── manifest.json
|   └── robots.txt
└── src
    ├── App.css
    ├── App.js
    ├── App.test.js
    ├── index.css
    ├── index.js
    ├── logo.svg
    └── serviceWorker.js
```

讓我們探索一下 **React** 項目的重要文件和文件夾。
對於要構建的項目，這些文件必須具有準確的文件名：

- **public/index.html** 是頁面模板；
- **src/index.js** 是 **JavaScript** 入口點。

您可以刪除或重命名其他文件 讓我們快速探索項目結構。

package.json - **package.json** 文件包含我們的 **React JS** 項目所需的所有依賴項。最重要的是，您可以檢查您正在使用的當前 **React** 版本。它包含啟動、構建和彈出 **React** 应用程序的所有腳本。

public 文件夾 - **public** 文件夾包含 **index.html**。由於 **React** 用於構建單頁面应用程序，因此我們使用這個 **HTML** 文件來渲染所有組件。基本上，它是一個 **HTML** 模板。它有一個以 **id** 作為 **根**的 **div** 元素，我們的所有組件都在這個 **div** 中呈現，並以 **index.html** 作為完整 **React** 应用程序的單個頁面。

src 文件夾- 在此文件夾中，我們擁有所有全局 **javascript** 和 **CSS** 文件。我們將要構建的所有不同組件都位於此處。

index.js - 這是您的 **React** 应用程序的頂級渲染器。

node_modules - **NPM** 或 **Yarn** 安裝的所有包都將駐留在 **node_modules** 文件夾中。

App.js - **App.js** 文件包含我們的 **App** 組件的定義，該組件實際上在瀏覽器中呈現，這是根組件。

2 - 使用 **NPM** 在 **React** 中添加 **Bootstrap**

打開一個新的終端窗口，導航到項目的文件夾，然後運行以下命令：

```
$ npm install bootstrap --save
```

安裝引導程序包後，您需要將其導入到 React 應用程序入口文件中。

打開 `src/index.js` 文件並添加以下程式碼：

```
import 'bootstrap/dist/css/bootstrap.min.css' ;
```

src/index.js

3.React 服務組件-REST API 調用

對於我們的 API 調用，我們將使用 `Axios`。以下是安裝 `Axios` 的 npm 命令。

```
npm install axios --save
```

下面是 `EmployeeService.js` 服務實現，用於通過 `Axios` 進行 HTTP REST 調用。

```
import axios from 'axios';

const EMPLOYEE_API_BASE_URL = 'http://localhost:8080/api/employees';

class EmployeeService{

  getEmployees(){
    return axios.get(EMPLOYEE_API_BASE_URL);
  }
}

export default new EmployeeService();
```

請注意，我們的後端 `Employee` 端點位

於 <http://localhost:8080/api/employees>。

確保創建 `EmployeeService` 類的對象並將其導出為：

```
export default new EmployeeService();
```

4. 開發 React 組件

組件是我們整個 React 应用程序的構建塊。它們就像接受 `props`、`state` 方面的輸入並輸出在瀏覽器中呈現的 UI 的函數。它們是可重用和可組合的。

React 組件可以是函數組件，也可以是類組件。在此示例中，我們將使用帶有 Reach Hooks 的功能組件。

讓我們創建一個 *EmployeeComponent.js* 文件並向其中添加以下代碼。

```
import React, {useState, useEffect} from 'react'
import EmployeeService from '../services/EmployeeService';

function EmployeeComponent() {

  const [employees, setEmployees] = useState([])

  useEffect(() => {
    getEmployees()
  }, [])

  const getEmployees = () => {

    EmployeeService.getEmployees().then((response) => {
      setEmployees(response.data)
      console.log(response.data);
    });
  };

  return (
    <div className = "container">

      <h1 className = "text-center"> Employees List</h1>

      <table className = "table table-striped">
        <thead>
          <tr>
            <th> Employee Id</th>
            <th> Employee First Name</th>
            <th> Employee Last</th>
```

```

        <th> Employee Email</th>
      </tr>

    </thead>
    <tbody>
      {
        employees.map(
          employee =>
            <tr key = {employee.id}>
              <td> {employee.id }</td>
              <td> {employee.firstName }</td>
              <td> {employee.lastName }</td>
              <td> {employee.email }</td>

            </tr>

          )
        }
      </tbody>

    </table>

  </div>
)
}

export default EmployeeComponent

```

讓我們了解上面代碼片段中使用的 React Hooks。

useState() Hooks

useState 是一個 Hook（函數），允許您在函式組件中擁有狀態變數。您將初始狀態傳遞給該函數，它返回一個具有當前狀態值（不一定是初始狀態）的變量以及另一個更新該值的函數。

```
const [employees, setEmployees] = useState([])
```

useEffect() Hooks

useEffect Hook 允許我們在函數組件執行 `useEffect()`。它不使用類別組件中可用的組件生命週期方法。換句話說，Effects Hooks 相當於 `componentDidMount()`、`componentDidUpdate()`和 `componentWillUnmount()`生命週期方法。

副作用具有大多數 Web 應用程序需要執行的共同功能，例如：

- 更新 DOM，
- 從服務器 API 獲取和使用數據，
- 設置訂閱等

範例：進行 REST API 調用：

```
useEffect(() => {
  getEmployees()
}, [])

const getEmployees = () => {

  EmployeeService.getEmployees().then((response) => {
    setEmployees(response.data)
    console.log(response.data);
  });
};
```

5. app.js

在上一步中，我們已經建了 `EmployeeComponent`，所以讓我們繼續將 `EmployeeComponent` 添加到 `App` 組件中：

```
import './App.css';
import EmployeeComponent from './components/EmployeeComponent';

function App() {
  return (
    <EmployeeComponent />
  );
};
```

```
}
```

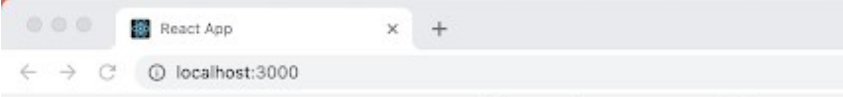
```
export default App;
```

6. 運行 React 應用程序

使用以下命令啟動項目：

```
npm start
```

在開發模式下運行應用程序。在瀏覽器中打開 <http://localhost:3000> 即可查看。



Employee Id	Employee First Name	Employee Last
1	Ramesh	Fadatare
2	Tony	Stark
3	John	Cena